

Proyecto Final Programación IV

Integrantes de Grupo

Jose Miguel Grisales Restrepo - 1055753117

Carlos Eduardo Grisales Restrepo - 1055750849

Información del Proyecto

- **Curso:** Programación I
- **Proyecto:** Sistema de Gestión Clínica
- **Tecnologías:** Python, Tkinter, MongoDB Atlas
- **Patrones aplicados:** MVC, Prototype, Template Method, Repository

1. CONTEXTO Y TEMÁTICA DEL DESARROLLO

Descripción General:

Sistema de gestión integral para clínicas médicas que permite administrar pacientes, doctores y citas médicas. El sistema utiliza una base de datos MongoDB para persistencia de datos y una interfaz gráfica desarrollada en Tkinter para interacción con el usuario.

Objetivo:

Facilitar la administración de información médica, incluyendo registro de pacientes, gestión de doctores, y programación de citas médicas con validaciones de disponibilidad.

2. REQUERIMIENTOS FUNCIONALES

RF-001: Gestión de Pacientes

El sistema debe ofrecer herramientas completas para la administración de pacientes dentro de la clínica. Para ello, contempla las siguientes funcionalidades:

- **RF-001.1:** El sistema debe permitir agregar nuevos pacientes, registrando información como nombre, documento, edad, género e historial médico.

- **RF-001.2:** El sistema debe permitir actualizar la información de pacientes existentes mediante la búsqueda por número de documento.
- **RF-001.3:** El sistema debe permitir eliminar pacientes del sistema de manera permanente.
- **RF-001.4:** El sistema debe permitir consultar todos los pacientes registrados, presentando la información en un formato de tabla.
- **RF-001.5:** El sistema debe generar automáticamente un **ID único** para cada paciente registrado.

RF-002: Gestión de Doctores

El sistema debe permitir manejar la información pertinente a los doctores de la clínica, incluyendo lo siguiente:

- **RF-002.1:** El sistema debe almacenar los datos de los doctores, tales como código, nombre, documento, especialidad, teléfono y correo electrónico.
- **RF-002.2:** El sistema debe permitir consultar la lista completa de doctores registrados en la base de datos.
- **RF-002.3:** El sistema debe cargar automáticamente la información de doctores desde la base de datos cuando la aplicación inicia.

RF-003: Gestión de Citas Médicas

El sistema debe facilitar la programación, validación y administración de citas médicas. Para ello incluye los siguientes requisitos:

- **RF-003.1:** El sistema debe permitir registrar citas médicas, asociando un paciente, un doctor, una fecha y una hora.
- **RF-003.2:** El sistema debe validar que el paciente exista antes de permitir la creación de una cita.
- **RF-003.3:** El sistema debe validar que el doctor no tenga otra cita programada en la misma fecha y hora.
- **RF-003.4:** El sistema debe validar que los formatos de fecha (YYYY-MM-DD) y hora (HH:MM) sean correctos.
- **RF-003.5:** El sistema debe eliminar automáticamente todas las citas asociadas a un paciente cuando este sea eliminado del sistema.

RF-004: Interfaz de Usuario

El sistema debe contar con una interfaz gráfica intuitiva y accesible para los usuarios administrativos de la clínica, cumpliendo con las siguientes características:

- **RF-004.1:** El sistema debe proporcionar una interfaz con un menú lateral que permita la navegación entre las distintas funciones.
- **RF-004.2:** El sistema debe centrar automáticamente la ventana principal en la pantalla al iniciar.
- **RF-004.3:** El sistema debe mostrar mensajes de confirmación y error mediante diálogos modales y notificaciones visuales.
- **RF-004.4:** El sistema debe abrir ventanas secundarias para ejecutar funciones específicas, como el registro de citas médicas.

3. REQUERIMIENTOS NO FUNCIONALES

RNF-001: Usabilidad

El sistema debe proporcionar una experiencia de usuario clara y accesible, permitiendo su utilización incluso por personal médico sin conocimientos técnicos.

- **RNF-001.1:** La interfaz debe ser intuitiva y fácil de navegar.
- **RNF-001.2:** Los formularios deben incluir validación de campos en tiempo real para reducir errores de registro.

RNF-002: Rendimiento

El rendimiento del sistema debe ser adecuado para garantizar una operación fluida durante el uso.

- **RNF-002.1:** El sistema debe cargar la lista completa de pacientes en un tiempo máximo de **2 segundos**.
- **RNF-002.2:** Las consultas a la base de datos deben ejecutarse en tiempo real, sin retrasos perceptibles para el usuario.

RNF-003: Confiabilidad

El sistema debe ser confiable y robusto, evitando inconsistencias o fallos inesperados.

- **RNF-003.1:** El sistema debe prevenir la duplicación de documentos de pacientes mediante el uso de **índices únicos** en la base de datos.
- **RNF-003.2:** Debe garantizar que no ocurran conflictos de horario al registrar citas médicas.
- **RNF-003.3:** Debe contar con manejo de excepciones para evitar cierres inesperados o pérdida de información.

RNF-004: Mantenibilidad

El sistema debe ser fácil de mantener y ampliar.

- **RNF-004.1:** El código debe estar estructurado de forma modular, separando cada funcionalidad en diferentes archivos.
- **RNF-004.2:** Debe utilizar patrones de diseño que permitan una futura expansión del sistema.
- **RNF-004.3:** El código crítico debe incluir comentarios y documentación adecuada.

RNF-005: Seguridad

La aplicación debe garantizar la integridad de los datos y la protección contra ataques comunes.

- **RNF-005.1:** Los campos numéricos deben contar con validación estricta de tipos de datos.
- **RNF-005.2:** Debe utilizar drivers oficiales de MongoDB para prevenir inyecciones y manipulaciones no autorizadas.

RNF-006: Compatibilidad

El sistema debe ser capaz de ejecutarse en diferentes entornos y comunicarse correctamente con la base de datos remota.

- **RNF-006.1:** Debe ser compatible con los sistemas operativos **Windows**, **macOS** y **Linux**, gracias al uso de Python y Tkinter.
- **RNF-006.2:** Debe permitir conexión con **MongoDB Atlas** en la nube para acceso remoto de los datos.

4. LIBRERÍAS IMPLEMENTADAS

Interfaz Gráfica

- `tkinter`
- `tkinter.ttk`
- `tkinter.messagebox`

Base de Datos

- `pymongo`
- `MongoClient`

Utilitarios

- `datetime`
- `uuid`
- `copy` (Prototype)
- `abc` (Template Method / Interfaces)
- `random`, `string`

5. FLUJO DE TRABAJO

Flujo principal

1. Se abre el sistema.
2. Usuario selecciona una operación:
 - Agregar paciente
 - Actualizar paciente
 - Eliminar paciente
 - Consultar pacientes
 - Registrar cita
 - Consultar doctores

Flujo para registrar una cita

1. Usuario abre ventana de citas.
2. Ingresar documento del paciente.

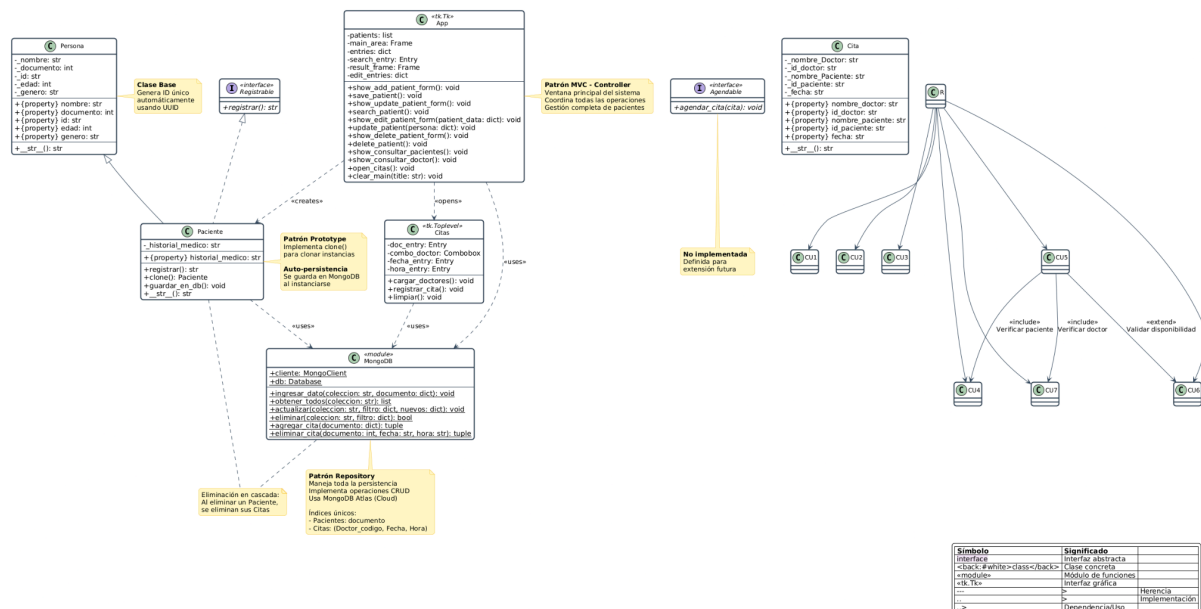
3. Selecciona médico.
4. Ingresa fecha/hora.
5. El sistema valida todo.
6. Inserta y muestra confirmación.

6. TARJETAS CRC

las tarjetas están **codificadas por colores en el archivo HTML**:

- **Gris oscuro:** Clases regulares (Persona, Paciente, Cita)
- **Morado:** Interfaces abstractas (Registrable, Agendable)
- **Verde:** Módulos de persistencia (MongoDB)
- **Rojo:** Interfaces gráficas (App, Citas)

7. DIAGRAMA DE CLASES



8. CASOS DE USO

CU-001 – Registrar Paciente

Actor: Recepcionista / Usuario

Precondiciones:

- El sistema debe estar en funcionamiento.
- El usuario debe estar en la ventana “Agregar Paciente”.

Flujo principal:

1. El usuario selecciona “Agregar Paciente” desde el menú lateral.
2. El sistema muestra el formulario de registro.
3. El usuario ingresa los datos del paciente.
4. El usuario confirma el registro presionando el botón “Guardar”.
5. El sistema valida los campos.
6. El sistema genera un ID único.
7. El sistema guarda el paciente en MongoDB.
8. El sistema muestra un mensaje confirmando el registro.

Flujos alternos:

- **A1:** Algún campo está vacío → El sistema muestra error y no guarda.
- **A2:** El documento ya existe → El sistema muestra error y cancela el registro.

Postcondiciones:

- El paciente queda registrado en la colección “Pacientes”.

CU-002 – Actualizar Información de Paciente

Actor: Recepcionista / Usuario

Precondiciones:

- Debe existir al menos un paciente registrado.
- El usuario debe conocer el documento del paciente.

Flujo principal:

1. El usuario hace clic en “Actualizar Paciente”.
2. El sistema muestra un campo para ingresar el documento.
3. El usuario digita el documento y busca.
4. El sistema recupera la información del paciente.
5. El usuario edita los campos requeridos.
6. El usuario confirma la actualización.
7. El sistema valida los datos.
8. El sistema actualiza los campos en la base de datos.
9. El sistema muestra un mensaje de éxito.

Flujos alternos:

- **A1:** Paciente no encontrado → Mostrar mensaje de error.
- **A2:** Campos inválidos → Mostrar error y no actualizar.

Postcondiciones:

- La información del paciente queda actualizada.

CU-003 – Eliminar Paciente

Actor: Recepcionista / Usuario

Precondiciones:

- El paciente debe existir en la base de datos.

Flujo principal:

1. El usuario selecciona "Eliminar Paciente".
2. El sistema solicita el documento del paciente.
3. El usuario ingresa el documento.
4. El sistema verifica la existencia del paciente.
5. El sistema muestra los datos del paciente y solicita confirmación.
6. El usuario confirma la eliminación.
7. El sistema elimina al paciente.
8. El sistema elimina también las citas asociadas.
9. El sistema muestra un mensaje de confirmación.

Flujos alternos:

- **A1:** Paciente no encontrado → Mostrar error.
- **A2:** Usuario cancela la operación → No se elimina nada.

Postcondiciones:

- El paciente y sus citas son eliminados permanentemente.

CU-004 – Consultar Pacientes

Actor: Recepcionista / Usuario

Precondiciones:

- Debe existir al menos un paciente registrado.

Flujo principal:

1. El usuario selecciona "Consultar Pacientes".
2. El sistema carga todos los pacientes desde MongoDB.
3. El sistema muestra los datos en una tabla con scroll.

Flujos alternos:

- **A1:** No hay pacientes registrados → Mostrar mensaje correspondiente.

Postcondiciones:

- Se presenta la lista de pacientes al usuario.

CU-005 – Registrar Cita Médica

Actor: Recepcionista / Usuario

Precondiciones:

- El paciente debe existir.
- El doctor debe existir.

Flujo principal:

1. El usuario selecciona “Agregar Cita”.
2. El sistema muestra formulario modal para registrar cita.
3. El usuario selecciona paciente.
4. El usuario selecciona doctor.
5. El usuario ingresa fecha y hora.
6. El sistema valida formato de fecha y hora.
7. El sistema verifica la disponibilidad del doctor.
8. El sistema guarda la cita.
9. El sistema muestra un mensaje de confirmación.

Flujos alternos:

- **A1:** Paciente inexistente → Error.
- **A2:** Doctor inexistente → Error.
- **A3:** Conflicto de horario → Error e impedir registro.

Postcondiciones:

- La cita se guarda en la colección “Citas”.

CU-006 – Consultar Citas Médicas

Actor: Recepcionista / Usuario

Precondiciones:

- Debe haber al menos una cita registrada.

Flujo principal:

1. El usuario selecciona “Consultar Citas”.
2. El sistema obtiene todas las citas almacenadas.
3. El sistema muestra la tabla con scroll vertical y horizontal.

Flujos alternos:

- **A1:** No existen citas registradas → Mostrar mensaje.

Postcondiciones:

- Se muestra la lista de citas al usuario.

CU-007 – Consultar Doctores

Actor: Recepcionista / Usuario

Precondiciones:

- Debe existir al menos un doctor registrado.

Flujo principal:

1. El usuario selecciona “Consultar Doctor”.
2. El sistema obtiene todos los doctores de la base de datos.
3. El sistema muestra la tabla con scroll.

Flujos alternos:

- **A1:** No existe información → Mostrar mensaje.

Postcondiciones:

- Se presenta la lista de doctores al usuario.

